Chapter 8: SQL – Additional (Joins & Subqueries)

Summary of slide deck — Multiple-table queries in SQL

Objectives

- Use joins to retrieve data from more than one table
- Use the IN and EXISTS operators to query multiple tables
- Use a subquery within a subquery (nested subqueries)
- Use table aliases for simplicity
- Join a table to itself
- Perform set operations: UNION, UNION ALL, INTERSECT, and MINUS/EXCEPT
- Use the ALL and ANY operators in a query
- Perform special join operations: inner join, outer join, and product (Cartesian product)

Querying Multiple Tables (Joins)

- When querying more than one table, the tables must be **joined**
- Tables are joined by finding columns with matching data (usually a primary key/foreign key pair)
- The join condition is placed in the WHERE clause

General steps to join two tables:

- SELECT clause — list all columns you want to display
- FROM clause — list all tables involved in the query
- WHERE clause — restrict results to rows with matching values in the common column(s)

Example: list each customer together with the sales rep who represents them — requires joining the CUSTOMER and REP tables on REP_NUM.

Comparing JOIN, IN, and EXISTS

- Tables can be related using a JOIN, or using the **IN** operator with a subquery, or the **EXISTS** operator
- IN checks whether a value appears in the result set of a subquery
- EXISTS checks whether a subquery returns any rows at all, and is often used to pull data from more than one table without a formal join

Correlated Subqueries

- A correlated subquery references a table listed in the outer (main) query
- The inner query's column must be qualified with its table name, e.g. ORDERS.ORDER_NUM
- For each row processed by the outer query, the subquery re-executes using that row's value

Nested Subqueries

- A nested subquery is a subquery placed within another subquery
- SQL evaluates queries from the innermost query outward to the outermost query

- More than one valid query formulation can often achieve the same result
- Most DBMS include optimizers that analyze queries and choose an efficient execution plan

Example — orders that include a part located in Warehouse 3, using nested subqueries:

```
SELECT ORDER_NUM, ORDER_DATE
FROM ORDERS
WHERE ORDER_NUM IN
  (SELECT ORDER_NUM
    FROM ORDER_LINE
    WHERE PART_NUM IN
      (SELECT PART_NUM
        FROM PART
        WHERE WAREHOUSE = '3'));
```

A query with nested subqueries is evaluated in 3 steps:

- Innermost subquery produces a temporary table of part numbers for parts located in the given warehouse
- Intermediate query produces a second temporary table listing order numbers that have an ORDER_LINE row referencing those part numbers
- Outer query produces the final list of order numbers and order dates, limited to orders found in the step-2 temporary table

The same result can also be written as a single query with an explicit multi-table join:

```
SELECT ORDERS.ORDER_NUM, ORDER_DATE
FROM ORDER_LINE, ORDERS, PART
WHERE ORDER_LINE.ORDER_NUM = ORDERS.ORDER_NUM
AND ORDER_LINE.PART_NUM = PART.PART_NUM
AND WAREHOUSE = '3';
```

HAVING and GROUP BY

- The HAVING clause restricts the output of a GROUP BY query
- It applies a conditional criterion to the grouped rows (unlike WHERE, which filters individual rows before grouping)

Using an Alias

- An alias is an alternate, shorter name for a table
- Used when tables are listed in the FROM clause
- Created by typing the table name, a space, then the alias name
- Simplifies writing and reading queries, especially when a table is joined to itself

Example — list each sales rep together with the customers they represent, using aliases R and C:

```
SELECT R.REP_NUM, LAST_NAME, FIRST_NAME, C.CUSTOMER_NUM, CUSTOMER_NAME
FROM REP R, CUSTOMER C
WHERE R.REP_NUM = C.REP_NUM;
```

Joining Several Tables

Step-by-step approach:

- In the SELECT clause, list all columns to display, qualifying column names where needed
- In the FROM clause, list all tables involved — including any table used only in the WHERE clause

- Take one pair of related tables at a time and state the join condition relating them in the WHERE clause
- Combine multiple join conditions with the AND operator
- Add any further filtering conditions, also joined with AND

Product (Cartesian Product)

- The product of two tables combines every row of the first table with every row of the second table
- Omitting the WHERE clause when selecting from two or more tables produces a product

Reference

Coronel, C., Morris, S. (2017). *Database Systems: Design, Implementation and Management*, 12th ed. Cengage Learning Boston.
Pratt, P.J. (2005). *A Guide to SQL*, 7th ed. Thomson/Course Technology.